Hochschule: Hochschule Darmstadt

Fachbereich: Media

Studiengang: Information Science

Projekt Webentwicklung

„PurrrfectMatch“

Veranstaltung: Projekt Webentwicklung

Dozent/in: Prof. Dr. Bernhard Thull, Prof. Dr. Elke Lang

Vorgelegt von: Kuberlinov, Denis

Matrikelnummer: 1121541

1. Data Flow Diagram / Datenflussdiagramm

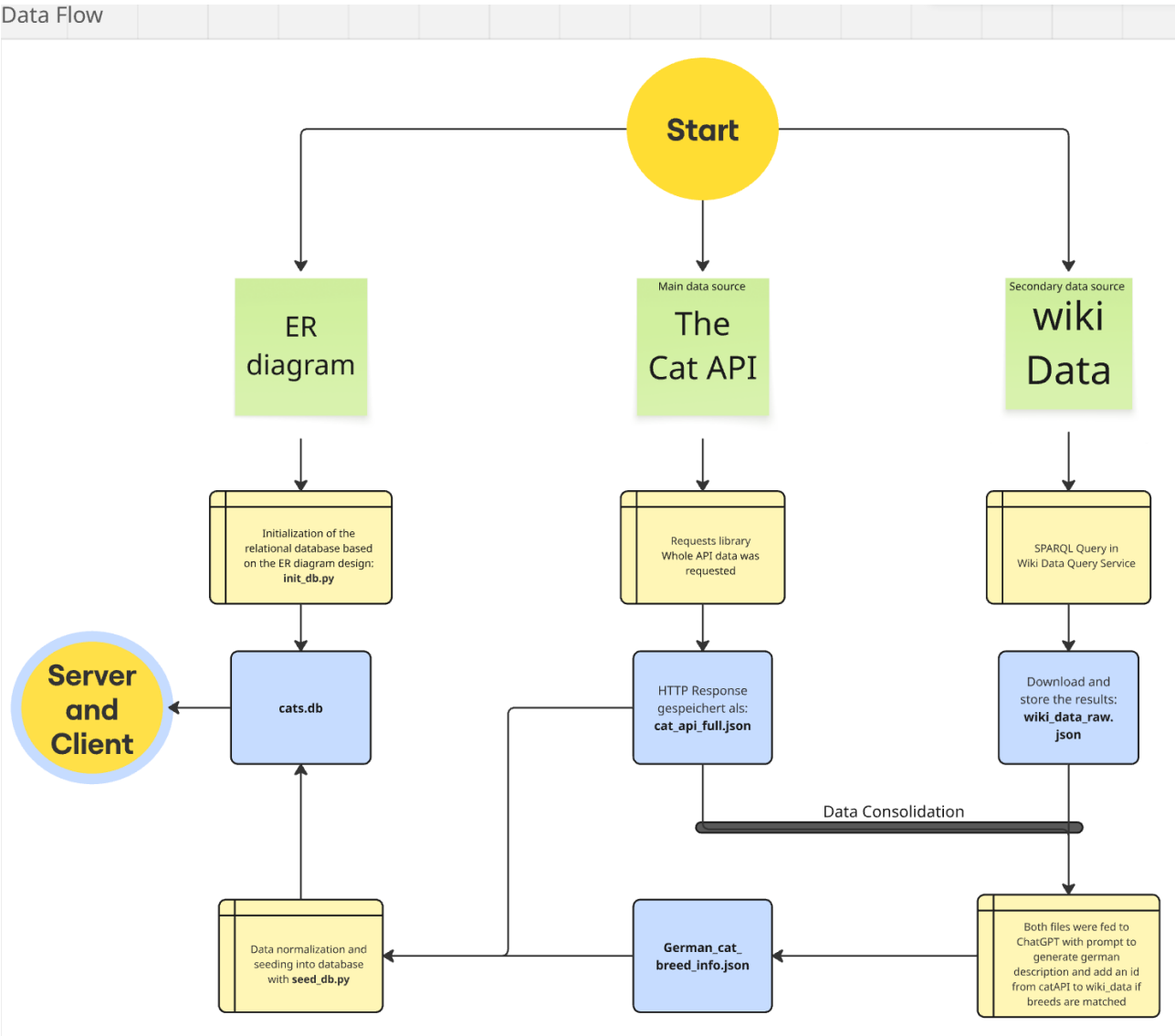

figure 1: data flow diagram

Data Sources

At the beginning, Wikidata and DBpedia were evaluated as the main data sources. A short test showed that both were missing too many consistently filled attributes to meet the project requirements.

In Wikidata I found roughly 90 cat breed entries in total, around 70 entries with an image, roughly 50 entries with a country of origin, and approximately 27 entries including breed weight information.

Even these fairly common properties were incomplete across breeds. Overall, the dataset was too sparse and inconsistent to use as the primary foundation of the application.

To get a reliable base dataset, The Cat API was chosen as the primary source. It offers a consistent set of roughly 70 breeds with rich descriptive metadata and several numeric fields that work well for filtering and comparisons.

Because The Cat API content is English-only and the assignment required at least two sources, Wikidata was kept as a secondary source. Its use was limited to German breed names, German descriptions, and links to German Wikipedia articles.

All other descriptive and numeric attributes were taken exclusively from The Cat API (an overview of the descriptive properties can be seen below in the figure 2: ER-diagram, cat_breeds table)

Reflections on the initial data flow diagram

Compared to the original plan, the overall data flow stayed the same. Data is fetched from two external sources, The Cat API and Wikidata. After extraction the datasets were consolidated so they can be loaded into a single relational model.

The consolidated data is seeded into a SQLite database. Because both sources deliver structured data, extensive cleaning was not required. Most transformations are lightweight normalizations, for example parsing ranged values such as metric_weight "3 - 5" and life_span "14 - 15" into min and max columns.

In practice, the backend follows a straightforward Extract-Transform-Load (ETL) flow.

A small deviation from a typical ETL workflow is the use of a ChatGPT-assisted consolidation step. It was used only to align identifiers between sources by adding the Cat API breed id to the Wikidata export, which made joining and seeding possible.

The detailed data pipeline description, along with the ETL process documentation and id alignment details, is available in the corresponding **Jupyter notebook** in the GitLab repository.

2. Entity Relationship Diagramm / ER-Diagramm

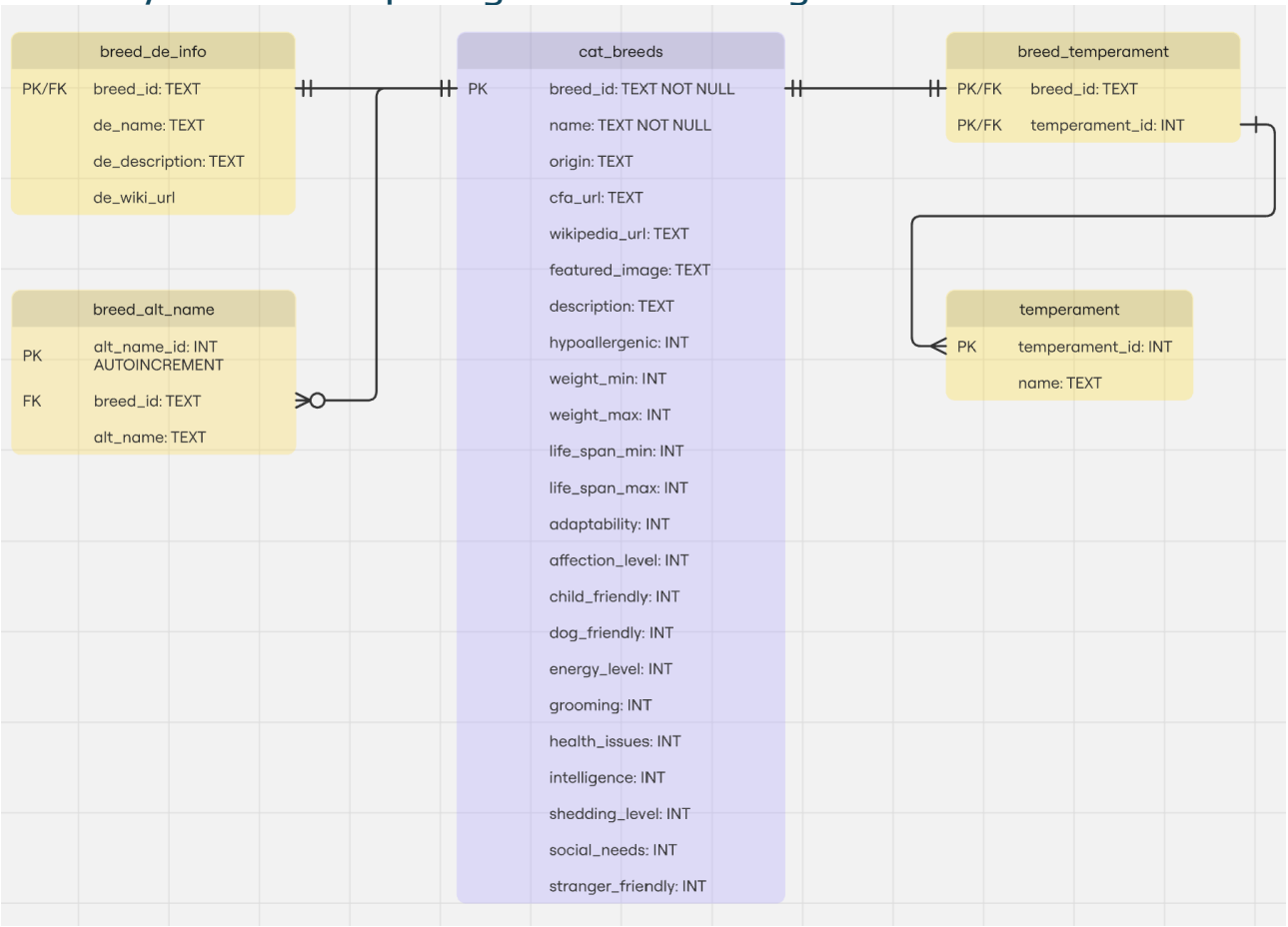

figure 2: final ER diagram

The data model is centered around the cat breed entity. Each record represents exactly one unique cat breed, which already satisfies the first normal form.

To ensure compliance with the third normal form, multivalued and repeating attributes were extracted into separate tables. These tables are connected to the main cat_breeds table via foreign keys, ensuring referential integrity.

Temperaments Table: Many-to-Many Relationship

Temperament traits represent shared categorical concepts that occur across many cat breeds.

Key characteristics:

- The same temperament traits (e.g. *Active*, *Gentle*, *Intelligent*) appear in many breeds
- Each breed can have multiple temperament traits
- Each temperament trait can be associated with multiple breeds

This constitutes a classic many-to-many relationship, which cannot be represented directly within a single relational table. To model this relationship correctly, a junction table (breed_temperament) was introduced.

The relationship is structured as follows:

- The temperament table stores each unique temperament trait exactly once and assigns it a dedicated primary key (temperament_id)
- The breed_temperament table links cat breeds and temperament traits by storing pairs of breed_id and temperament_id
- The combination of breed_id and temperament_id forms a composite primary key in the junction table, ensuring that no duplicate breed–temperament relationships can be stored

Both columns in the junction table function simultaneously as foreign keys referencing their parent tables. This design enforces referential integrity and guarantees that:

- a temperament relationship cannot exist without a valid breed
- a temperament trait must exist before it can be assigned to a breed

During the database seeding process, each unique temperament description was assigned a unique identifier.

Alternative Names Table: One-to-Many Relationship

Alternative breed names differ fundamentally from temperament traits and therefore require a different modeling approach.

Key characteristics:

- An alternative name belongs to exactly one breed
- Alternative names are not shared across breeds
- They have no independent semantic meaning outside their associated breed

For example, the breed *Birman* may have alternative names such as *Sacred Birman* or *Sacred Cat of Burma*. These names are never reused for other breeds.

This relationship is therefore one-to-many:

- One breed → many alternative names

Alternative names are stored in a dependent table that includes:

- a foreign key (breed_id) referencing the primary key of the cat_breeds table
- the alternative name as a descriptive attribute

The combination of breed_id and the alternative name uniquely identifies each record, preventing duplicate alternative names for the same breed. Since alternative names are not shared and do not require independent identification, no separate primary key or junction table is necessary.

3. Schema Simplification

During development, the initial ER diagram underwent several simplifications based on feasibility and scope

- A planned coat_type table was dropped, as its implementation would have required extensive workarounds and additional data sources beyond the project's constraints.
- It was decided to fetch and load additional images directly from The Cat API at runtime. Since a featured image describes exactly one breed, a separate image table was deemed unnecessary. Only a single featured image URL is stored in the database per breed.
- CFA recognition is represented as a descriptive attribute within the cat_breeds table rather than a separate entity.

While these simplifications reduce the app's scalability – for example, the ability to add additional breed associations or fetch gallery photos if catAPI is unavailable or behind a paywall and the uneven number of photos in the gallery depending on the rarity of the breed – they are sufficient for the prototype and keep the data model and implementation manageable.

4. Application Features and Filters

Based on the final database schema, the web application provides an overview page that supports search, sorting, and filters. All filters are implemented as query parameters in GET requests so they can be combined and bookmarked.

Sorting Filters

- Alphabetical sorting (A–Z)
- Weight: smallest to largest
- Weight: largest to smallest
- Life expectancy: longest to shortest

Checkbox Filters

- Breeds requiring grooming (grooming score ≥ 4)
- CFA-recognized breeds
- Hypoallergenic breeds
- Breeds that have german article (description and name)

Search

- Search by breed name (both german and english)

Breed Characteristics Panel:

- Most descriptive values from the main cat breeds table were converted into a dynamic breed characteristics table, which renders depending on the breed ID called.

figure 3: dynamic characteristics panel example

Dynamic Temperament Traits and Alternative Names:

- Each temperament trait received its own id which allowed a join and dynamic rendering depending on the requested breed.

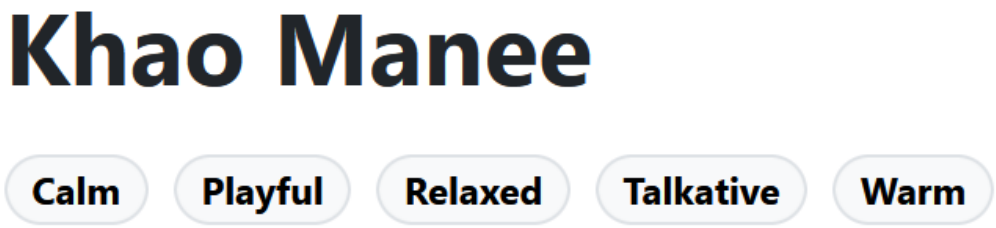

figure 4: dynamic temperament traits example

5. Lessons learned

This project was a major step in understanding how a complete local web application is built end-to-end – from raw data to a working UI.

Data pipeline & workflow

- A stable project needs a clear order: clean/structured data → database schema → seeding → server routes → templates/UI.
- Once the database became consistent and repeatable, features like filtering, sorting, and search were much easier to implement.

Relational database design (ERD → implementation)

- Learned how ER-diagram decisions affect real code: choosing primary keys, using foreign keys, and when junction tables are required (e.g., many-to-many temperament traits).
- Understood practical data constraints: optional fields, and the difference between NULL vs empty strings, how that impacts filtering/logic and the falsy/truthy python concept.

Database seeding (hardest part) – seeding required engineering, not just inserting data:

- correct import order (parent tables before dependent tables)
- idempotent seeding (INSERT OR IGNORE vs ON CONFLICT DO UPDATE)
- data preparation: splitting CSV-like fields, parsing ranges, cleaning values

Managing many entries emphasized the need for reliable, repeatable normalization scripts.

Flask fundamentals

- Learned core routing patterns including dynamic routes like /breeds/<breed_id>.
- Learned how query strings (?q=...) enable search/filter/sort on one overview page.

- Understood how Flask handles requests and passes data into templates for server-side rendering.

Jinja templating & app structure

- Built reusable UI with template inheritance (base.html) and partials (navbar/footer).
- Learned to use Blueprints to organize routes and keep the codebase maintainable as features grow.

Debugging & configuration

- A key lesson: SQLite errors like “no such table” often mean the app is connected to the wrong database file (especially due to Flask’s instance/ path vs seeding script paths).
- This reinforced the importance of a single source of truth for DB paths and consistent config across init/seed/runtime.

Python packages

- Learned what makes a folder a package (__init__.py) and why execution context matters for imports.
- Understood how import errors can occur depending on where scripts are run from (project root vs scripts folder), and why consistent structure/entry points matter.

Git version control & collaboration

- Practiced essential workflows: branches, merge requests, resolving merge conflicts, and core Git commands for collaborative development.

Future development ideas:

- This project now has a strong foundation for features that are common in real applications:
- CRUD functionality – admin interface for editing breeds / adding local content
- Caching API calls – store gallery image URLs temporarily or persist them
- Deployment – how to move from local SQLite to deployed environment (and what changes)
- Stronger search – case-insensitive search, matching alt names, optional full-text search (FTS)

Sources:

- Python and Flask: Web Development from Scratch
- Create A Flask Blog - Flask Friday
- Learn Flask for Python - Full Tutorial
- The Web Developer Bootcamp 2026
- Datacamp courses

- Flask Tutorialspoint
- SQLite tutorialspoint
- ChatGPT
- Bootstrap documentation